Chapter 5
Robust Defense Mechanisms

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 27, 2026

Contents

Setting and notation **1**

1 Adversarial Training **2**
1.1 The min-max formulation . . . 2
1.2 Variations . . . 2
1.3 Costs and limitations . . . 2

2 Certified Defenses: Randomized Smoothing **2**
2.1 The smoothed classifier . . . 3
2.2 Prediction and certification in practice . . . 3
2.3 Training for smoothing and limitations . . . 3

3 Input Transformation Defenses **3**

4 Gradient Masking and Obfuscation **4**
4.1 Circumvention techniques . . . 4

5 Defending Against Poisoning and Backdoors **4**
5.1 Data sanitization . . . 4
5.2 Robust training . . . 5
5.3 Model inspection and repair . . . 5

6 Hands-On: Implementing Adversarial Training **5**
6.1 PGD and fast adversarial training . . . 5
6.2 TRADES loss . . . 6
6.3 Randomized smoothing certification . . . 6
6.4 Suggested experiments . . . 6

Exercises **6**

Setting and notation

We reuse the notation of Chapters 1–3. The defender's objects are the training procedure, the model $f_{\boldsymbol{\theta}}$, and possibly a preprocessing map $g : \mathcal{X} \to \mathcal{X}$ applied before classification. A defense is only meaningful together with a threat model (Chapter 1): for evasion, typically $\mathcal{B}_p(\mathbf{x}, \epsilon)$ with a white-box adaptive attacker; for poisoning, a poisoning rate γ and a trigger or perturbation family. We distinguish *empirical* defenses, evaluated by running attacks (which only gives an upper bound on robust accuracy), from *certified* defenses, which give a proven lower bound.

1 Adversarial Training

1.1 The min-max formulation

Adversarial training minimizes the adversarial risk of Chapter 1 (Madry et al., 2018):

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\Big[\max_{\boldsymbol{\delta}\in\mathcal{B}_p(0,\epsilon)} \mathcal{L}(\boldsymbol{\theta};\mathbf{x}+\boldsymbol{\delta},y)\Big]. \tag{1}$$

The inner maximization is the attack; the outer minimization is training. In practice the inner problem is approximated by PGD with K steps, and the outer by SGD on the resulting adversarial examples.

Why train on the approximate maximizer? Danskin's theorem says that, under regularity conditions, the gradient of $\phi(\boldsymbol{\theta}) = \max_{\boldsymbol{\delta}} \mathcal{L}(\boldsymbol{\theta};\mathbf{x}+\boldsymbol{\delta},y)$ is $\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta};\mathbf{x}+\boldsymbol{\delta}^\star,y)$ at a maximizer $\boldsymbol{\delta}^\star$. Deep networks violate the assumptions (non-unique maximizers, non-smoothness), but the principle explains why using the loss gradient at the PGD solution is a sensible descent direction for the robust objective.

Algorithm 1 PGD adversarial training (one epoch)

for each minibatch (X, Y) **do**
  $X' \leftarrow \mathrm{PGD}_K(f_{\boldsymbol{\theta}}, X, Y, \epsilon, \alpha)$ ▷ model in eval mode for the attack, or consistent BN handling
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\nabla_{\boldsymbol{\theta}} \frac{1}{|X|}\sum \mathcal{L}(\boldsymbol{\theta}; X', Y)$
end for

1.2 Variations

TRADES. Zhang et al. (2019) decompose robust error into natural error plus boundary error and optimize a surrogate:

$$\min_{\boldsymbol{\theta}} \mathbb{E}\Big[\mathcal{L}_{\mathrm{CE}}(F(\mathbf{x}),y) + \beta \max_{\mathbf{x}'\in\mathcal{B}_p(\mathbf{x},\epsilon)} \mathrm{KL}\big(F(\mathbf{x}) \,\|\, F(\mathbf{x}')\big)\Big]. \tag{2}$$

The first term fits clean data; the second pushes the decision boundary away from the data by making predictions locally constant. β controls the accuracy-robustness trade-off explicitly.

Fast adversarial training. PGD-K training costs about $K+1$ times standard training. "Free" adversarial training (Shafahi et al., 2019) recycles the backward pass to update both the perturbation and the weights. Wong et al. (2020) showed that single-step FGSM training with a *random start* and a step size of about 1.25ϵ matches PGD training, provided one guards against *catastrophic overfitting*: a sudden collapse in which PGD accuracy drops to zero while FGSM accuracy stays high. Monitoring PGD accuracy on a validation batch and early stopping are essential.

Robust overfitting and data. Rice et al. (2020) found that, unlike standard training, robust test accuracy peaks shortly after the first learning-rate decay and then declines while robust training accuracy keeps increasing. Early stopping on robust validation accuracy recovers most of the gap. More data helps substantially: unlabeled data with pseudo-labels (Carmon et al., 2019) and, more recently, millions of images from diffusion models (Wang et al., 2023) produce the strongest ℓ_∞ results on CIFAR-10.

1.3 Costs and limitations

- **Accuracy trade-off.** Robust models have lower clean accuracy. Tsipras et al. (2019) give examples where robustness and accuracy are provably at odds, though in practice the gap shrinks with more data and capacity.
- **Threat model specificity.** Training against ℓ_∞ gives limited robustness to ℓ_2, ℓ_1 or spatial perturbations.
- **Computation.** Several times the cost of standard training.
- **No guarantee.** Robust accuracy measured with PGD is an upper bound. Evaluate with AutoAttack (Croce and Hein, 2020) and adaptive attacks (Chapter 6).

2 Certified Defenses: Randomized Smoothing

Empirical defenses can always be broken by a better attack. Certified defenses prove, for each input, a radius R such that no perturbation with $\|\boldsymbol{\delta}\| \le R$ changes the prediction. Exact verification of a ReLU network is NP-hard; relaxation-based methods (Wong and Kolter, 2018; Gowal et al., 2019) scale only to moderately sized networks. Randomized smoothing scales to ImageNet.

2.1 The smoothed classifier

Given any base classifier f and noise level σ, define

$$g(\mathbf{x}) = \arg\max_{c \in \mathcal{Y}} \Pr_{\boldsymbol{\eta} \sim \mathcal{N}(0, \sigma^2 I)} \big[f(\mathbf{x} + \boldsymbol{\eta}) = c \big]. \tag{3}$$

g returns the class that f predicts most often under Gaussian noise.

Theorem 2.1 (Cohen et al., 2019)**.** *Let c_A be the most likely class for g at $\mathbf{x}$, with $\Pr[f(\mathbf{x}+\boldsymbol{\eta}) = c_A] \geq \underline{p_A}$ and $\max_{c \neq c_A} \Pr[f(\mathbf{x}+\boldsymbol{\eta}) = c] \leq \overline{p_B}$. Then $g(\mathbf{x}+\boldsymbol{\delta}) = c_A$ for all $\|\boldsymbol{\delta}\|_2 < R$, with*

$$R = \frac{\sigma}{2}\Big(\Phi^{-1}(\underline{p_A}) - \Phi^{-1}(\overline{p_B})\Big),$$

where Φ^{-1} is the standard Gaussian quantile function. Using $\overline{p_B} = 1 - \underline{p_A}$ gives $R = \sigma\, \Phi^{-1}(\underline{p_A})$.

Intuition. Shifting the input by $\boldsymbol{\delta}$ shifts the Gaussian. The worst case for the defender is that f is a linear classifier aligned with $\boldsymbol{\delta}$; the Neyman–Pearson lemma shows that this worst case changes the class probabilities by an amount controlled by $\|\boldsymbol{\delta}\|_2/\sigma$. The guarantee holds *regardless of the base classifier*, which can be an arbitrary deep network. Lecuyer et al. (2019) derived a related, looser guarantee from differential privacy.

2.2 Prediction and certification in practice

The probabilities cannot be computed exactly, so they are estimated by Monte Carlo:

1. Draw n_0 noisy samples to select a candidate $\hat{c}_A$.
2. Draw n fresh samples and count n_A hits for $\hat{c}_A$.
3. Compute a one-sided $(1-\alpha)$ Clopper–Pearson lower bound $\underline{p_A}$ from n_A/n.
4. If $\underline{p_A} > 1/2$, return $\hat{c}_A$ with radius $R = \sigma\, \Phi^{-1}(\underline{p_A})$; otherwise abstain.

The certificate is probabilistic: with probability at least $1-\alpha$ over the sampling, the stated radius is correct. Note the ceiling: with n samples, $\underline{p_A} < 1$, so R is bounded (for $n = 10^5$ and $\alpha = 10^{-3}$, $R \leq \sigma\, \Phi^{-1}(\alpha^{1/n}) \approx 3.8\sigma$).

2.3 Training for smoothing and limitations

The base classifier must be accurate under Gaussian noise, so it is trained with noise augmentation at the same σ; adversarially training the smoothed classifier improves certified radii further (Salman et al., 2019). Off-the-shelf diffusion denoisers followed by a pretrained classifier give strong certified results without retraining (Carlini et al., 2023).

Limitations: the certificate is for ℓ_2 (certified ℓ_∞ radii from ℓ_2 are weak in high dimension, since $\|\boldsymbol{\delta}\|_2 \leq \sqrt{d}\|\boldsymbol{\delta}\|_\infty$); σ trades clean accuracy against radius; inference requires thousands of forward passes per input; and the certificate applies to g, not to f.

3 Input Transformation Defenses

Input transformations apply $g : \mathcal{X} \to \mathcal{X}$ before classification, hoping to remove adversarial structure while preserving content: JPEG compression, bit-depth reduction and median filtering (Xu et al., 2018), total variation minimization and image quilting (Guo et al., 2018), projection onto the range of a generative model (Samangouei et al., 2018), and, more recently, diffusion-based purification (Nie et al., 2022). The same transformations can be used for *detection*: flag inputs whose prediction changes a lot after squeezing (Xu et al., 2018).

Why they fail against adaptive attackers. The attacker simply attacks the composed classifier $f \circ g$. Difficulties in doing so are technical, not fundamental:

- non-differentiable g (quantization, JPEG): use BPDA (next section);
- randomized g: use Expectation over Transformation;
- iterative g (optimization or diffusion): differentiate through it, approximate it, or attack the full stochastic pipeline with many gradient samples.

Athalye et al. (2018a) broke most preprocessing defenses of ICLR 2018 this way. Diffusion purification remains an area where evaluation is difficult and claims must be checked with full adaptive attacks.

4 Gradient Masking and Obfuscation

Definition 4.1 (Gradient masking)**.** A defense exhibits gradient masking when it makes gradient-based attacks fail without making the model correct on the adversarial inputs that exist. The model is not robust; the attacker's optimizer is blinded.

Athalye et al. (2018a) identify three forms:

1. **Shattered gradients**: non-differentiable or numerically unstable operations (quantization, thresholding).
2. **Stochastic gradients**: randomized defenses where single-sample gradients are noisy.
3. **Vanishing or exploding gradients**: very deep computation (iterative purification, recurrent unrolling) or saturated softmax outputs, as in defensive distillation (Papernot et al., 2016).

4.1 Circumvention techniques

BPDA (Backward Pass Differentiable Approximation). If g is non-differentiable but $g(\mathbf{x}) \approx \mathbf{x}$, use g in the forward pass and the identity (or another differentiable approximation $\tilde{g}$) in the backward pass:

$$\nabla_{\mathbf{x}} \mathcal{L}\big(f(g(\mathbf{x})), y\big) \;\approx\; \nabla_{\mathbf{z}} \mathcal{L}\big(f(\mathbf{z}), y\big)\big|_{\mathbf{z}=g(\mathbf{x})}.$$

EOT (Expectation over Transformation). For randomized defenses g_r with random seed r, maximize the expected loss, estimating its gradient by averaging (Athalye et al., 2018b):

$$\nabla_{\mathbf{x}} \, \mathbb{E}_r\big[\mathcal{L}(f(g_r(\mathbf{x})), y)\big] \approx \frac{1}{M} \sum_{m=1}^{M} \nabla_{\mathbf{x}} \mathcal{L}\big(f(g_{r_m}(\mathbf{x})), y\big).$$

Reparameterization. For iterative or vanishing-gradient defenses, write $\mathbf{x} = h(\mathbf{z})$ with h such that $g(h(\mathbf{z})) = h(\mathbf{z})$, and optimize over $\mathbf{z}$. Using logits or margin losses instead of probabilities avoids softmax saturation.

Red flags for gradient masking (Carlini et al., 2019)

- One-step attacks perform better than iterative attacks.
- Black-box or transfer attacks perform better than white-box attacks.
- Unbounded attacks ($\epsilon \to \infty$) do not reach 100% success.
- Random sampling within the ball finds adversarial examples that gradient attacks miss.
- Increasing iterations or restarts does not increase success.

Gradient-free attacks such as Square Attack or SPSA (Chapter 2) are good diagnostics because masking does not affect them.

5 Defending Against Poisoning and Backdoors

Defenses act at three points: on the data before training, during training, and on the trained model.

5.1 Data sanitization

Outlier removal. Remove points far from their class centroid or with high loss under a reference model. For convex learners, Steinhardt et al. (2017) analyze the worst case of sphere and slab defenses. Clean-label poisons are designed to be in-distribution, so simple outlier detection often misses them.

Spectral signatures. Tran et al. (2018) observe that, in the representation $\mathbf{h}(\mathbf{x})$ of a backdoored network, poisoned samples of the target class shift the class mean along a particular direction. For each class, center the representations, compute the top right singular vector $\mathbf{v}$ of the centered matrix, score each sample by $(\mathbf{v}^\top(\mathbf{h}(\mathbf{x}) - \bar{\mathbf{h}}))^2$, remove the highest-scoring samples, and retrain.

Activation clustering. Chen et al. (2019) cluster the representations of each class into two groups (after dimensionality reduction) and flag a class whose smaller cluster is suspiciously large and well separated.

5.2 Robust training

- **Partition and vote.** Deep Partition Aggregation (Levine and Feizi, 2021) trains k models on disjoint, hash-determined partitions of the data and predicts by majority vote. Each poisoned sample affects at most one model, so the prediction is certifiably unchanged if the vote margin exceeds twice the number of poisoned samples.
- **Differential privacy.** DP-SGD bounds the influence of any single sample, which limits small-γ targeted poisoning, at the cost of accuracy.
- **Robust aggregation in federated learning.** Replace averaging with Krum (Blanchard et al., 2017), coordinate-wise median or trimmed mean (Yin et al., 2018), or norm clipping of updates. These limit, but do not eliminate, backdoor injection by a minority of clients (Bagdasaryan et al., 2020).

5.3 Model inspection and repair

Trigger reverse engineering. Neural Cleanse (Wang et al., 2019) searches, for each candidate target class c, for the smallest mask $\mathbf{M}$ and pattern $\mathbf{p}$ that send clean inputs to c:

$$\min_{\mathbf{M},\mathbf{p}} \sum_{\mathbf{x}} \mathcal{L}\big(f((\mathbf{1}-\mathbf{M})\odot\mathbf{x}+\mathbf{M}\odot\mathbf{p}), c\big) + \lambda\|\mathbf{M}\|_1.$$

A class requiring an anomalously small mask (detected by the median absolute deviation across classes) is flagged as backdoored, and the reversed trigger is used to unlearn it.

Fine-pruning. Liu et al. (2018) prune neurons that are dormant on clean data (where backdoors often hide) and then fine-tune on a small clean set.

Run-time detection. STRIP (Gao et al., 2019) superimposes an input with random clean images and measures the entropy of the predictions; triggered inputs keep predicting the target class, giving abnormally low entropy.

Adaptive poisoning

Each of these defenses relies on a property of known attacks: poisons are outliers, triggers are small, backdoor features are separable. Attacks have been designed to violate each assumption (sample-specific and dynamic triggers, latent-separability-aware poisoning, clean-label poisons). Evaluate poisoning defenses against attackers who know the defense, as for evasion.

6 Hands-On: Implementing Adversarial Training

6.1 PGD and fast adversarial training

```
import torch, torch.nn.functional as F

def pgd(model, x, y, eps, alpha, steps):
    delta = torch.empty_like(x).uniform_(-eps, eps)
    delta = ((x + delta).clamp(0, 1) - x).detach()
    for _ in range(steps):
        delta.requires_grad_(True)
        loss = F.cross_entropy(model(x + delta), y)
        g, = torch.autograd.grad(loss, delta)
        delta = (delta + alpha * g.sign()).clamp(-eps, eps)
        delta = ((x + delta).clamp(0, 1) - x).detach()
    return (x + delta).detach()

def train_epoch(model, loader, opt, eps=8/255, alpha=2/255, steps=10,
                fast=False):
    for x, y in loader:
        x, y = x.cuda(), y.cuda()
        model.eval()                        # attack with fixed BN statistics
        if fast:                            # FGSM with random start
            x_adv = pgd(model, x, y, eps, alpha=1.25 * eps, steps=1)
        else:
            x_adv = pgd(model, x, y, eps, alpha, steps)
        model.train()
        loss = F.cross_entropy(model(x_adv), y)
        opt.zero_grad(); loss.backward(); opt.step()
```

6.2 TRADES loss

```
def trades_loss(model, x, y, eps, alpha, steps, beta=6.0):
    model.eval()
    with torch.no_grad():
        p_nat = F.softmax(model(x), 1)
    x_adv = x + 0.001 * torch.randn_like(x)
    for _ in range(steps):
        x_adv.requires_grad_(True)
        kl = F.kl_div(F.log_softmax(model(x_adv), 1), p_nat,
                      reduction="batchmean")
        g, = torch.autograd.grad(kl, x_adv)
        x_adv = x_adv.detach() + alpha * g.sign()
        x_adv = torch.min(torch.max(x_adv, x - eps), x + eps).clamp(0, 1)
    model.train()
    logits = model(x)
    robust = F.kl_div(F.log_softmax(model(x_adv), 1), F.softmax(logits, 1),
                      reduction="batchmean")
    return F.cross_entropy(logits, y) + beta * robust
```

6.3 Randomized smoothing certification

```
from scipy.stats import beta as Beta, norm

@torch.no_grad()
def sample_counts(model, x, sigma, n, K, bs=1000):
    counts = torch.zeros(K, dtype=torch.long)
    for i in range(0, n, bs):
        m = min(bs, n - i)
        noisy = x.repeat(m, 1, 1, 1) + sigma * torch.randn(m, *x.shape[1:],
                                                           device=x.device)
        counts += torch.bincount(model(noisy).argmax(1).cpu(), minlength=K)
    return counts

def certify(model, x, sigma, n0=100, n=100_000, alpha=1e-3, K=10):
    cA = sample_counts(model, x, sigma, n0, K).argmax().item()
    nA = sample_counts(model, x, sigma, n, K)[cA].item()
    pA = Beta.ppf(alpha, nA, n - nA + 1)      # Clopper-Pearson lower bound
    if pA <= 0.5:
        return None, 0.0                       # abstain
    return cA, sigma * norm.ppf(pA)
```

6.4 Suggested experiments

1. Train PreAct ResNet-18 on CIFAR-10 with standard training, PGD-10 adversarial training, fast FGSM training and TRADES ($\beta \in \{1, 6\}$), all at $\epsilon = 8/255$. Report clean accuracy, PGD-50 accuracy, AutoAttack accuracy and training time.
2. Track PGD robust accuracy on a validation set every epoch for PGD training over 200 epochs with a piecewise learning rate. Reproduce robust overfitting and evaluate early stopping.
3. Run fast FGSM training with step sizes 1.25ϵ and 2ϵ. Detect catastrophic overfitting from the training curves.
4. Train noise-augmented base classifiers at $\sigma \in \{0.12, 0.25, 0.5\}$ and plot certified accuracy versus ℓ_2 radius for the smoothed classifiers.
5. Wrap a standard model with bit-depth reduction (3 bits). Attack it with plain PGD and with PGD+BPDA. Explain the difference.

Exercises

1. State Danskin's theorem and explain which of its assumptions fail for ReLU networks. Why does PGD adversarial training still work in practice?
2. For a linear base classifier $f(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^\top \mathbf{x} + b)$, compute $\Pr[f(\mathbf{x} + \boldsymbol{\eta}) = 1]$ under $\boldsymbol{\eta} \sim \mathcal{N}(0, \sigma^2 I)$ and show that the randomized smoothing radius equals the distance from $\mathbf{x}$ to the hyperplane.
3. With $n = 10^4$ samples, all of which return class c_A, and $\alpha = 10^{-3}$, compute the certified radius for $\sigma = 0.25$. What is the maximum radius you could certify with this n?
4. A defense quantizes inputs to 4 bits and reports 70% robust accuracy under PGD at $\epsilon = 8/255$. Describe how to evaluate it properly and predict the outcome.

5. Show that in Deep Partition Aggregation with k partitions, if the top class has n_1 votes and the runner-up has n_2 votes, the prediction cannot change when at most $\lfloor (n_1 - n_2)/2 \rfloor$ training samples are poisoned (ignoring tie-breaking).

References

Athalye, A., Carlini, N., and Wagner, D. (2018a). Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 274–283.

Athalye, A., Engstrom, L., Ilyas, A., and Kwok, K. (2018b). Synthesizing robust adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 284–293.

Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., and Shmatikov, V. (2020). How to backdoor federated learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 2938–2948.

Blanchard, P., El Mhamdi, E. M., Guerraoui, R., and Stainer, J. (2017). Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A., and Kurakin, A. (2019). On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*.

Carlini, N., Tramèr, F., Dvijotham, K. D., Rice, L., Sun, M., and Kolter, J. Z. (2023). (certified!!) adversarial robustness for free! In *International Conference on Learning Representations (ICLR)*.

Carmon, Y., Raghunathan, A., Schmidt, L., Liang, P., and Duchi, J. C. (2019). Unlabeled data improves adversarial robustness. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Chen, B., Carvalho, W., Baracaldo, N., Ludwig, H., Edwards, B., Lee, T., Molloy, I., and Srivastava, B. (2019). Detecting backdoor attacks on deep neural networks by activation clustering. In *Workshop on Artificial Intelligence Safety, AAAI*.

Cohen, J., Rosenfeld, E., and Kolter, J. Z. (2019). Certified adversarial robustness via randomized smoothing. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 1310–1320.

Croce, F. and Hein, M. (2020). Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2206–2216.

Gao, Y., Xu, C., Wang, D., Chen, S., Ranasinghe, D. C., and Nepal, S. (2019). STRIP: A defence against trojan attacks on deep neural networks. In *Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC)*, pages 113–125.

Gowal, S., Dvijotham, K., Stanforth, R., Bunel, R., Qin, C., Uesato, J., Arandjelovic, R., Mann, T., and Kohli, P. (2019). Scalable verified training for provably robust image classification. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4842–4851.

Guo, C., Rana, M., Cisse, M., and van der Maaten, L. (2018). Countering adversarial images using input transformations. In *International Conference on Learning Representations (ICLR)*.

Lecuyer, M., Atlidakis, V., Geambasu, R., Hsu, D., and Jana, S. (2019). Certified robustness to adversarial examples with differential privacy. In *IEEE Symposium on Security and Privacy (S&P)*, pages 656–672.

Levine, A. and Feizi, S. (2021). Deep partition aggregation: Provable defenses against general poisoning attacks. In *International Conference on Learning Representations (ICLR)*.

Liu, K., Dolan-Gavitt, B., and Garg, S. (2018). Fine-pruning: Defending against backdooring attacks on deep neural networks. In *Research in Attacks, Intrusions, and Defenses (RAID)*, pages 273–294.

Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*.

Nie, W., Guo, B., Huang, Y., Xiao, C., Vahdat, A., and Anandkumar, A. (2022). Diffusion models for adversarial purification. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pages 16805–16827.

Papernot, N., McDaniel, P., Wu, X., Jha, S., and Swami, A. (2016). Distillation as a defense to adversarial perturbations against deep neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 582–597.

Rice, L., Wong, E., and Kolter, J. Z. (2020). Overfitting in adversarially robust deep learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 8093–8104.

Salman, H., Li, J., Razenshteyn, I., Zhang, P., Zhang, H., Bubeck, S., and Yang, G. (2019). Provably robust deep learning via adversarially trained smoothed classifiers. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Samangouei, P., Kabkab, M., and Chellappa, R. (2018). Defense-GAN: Protecting classifiers against adversarial attacks using generative models. In *International Conference on Learning Representations (ICLR)*.

Shafahi, A., Najibi, M., Ghiasi, A., Xu, Z., Dickerson, J., Studer, C., Davis, L. S., Taylor, G., and Goldstein, T. (2019). Adversarial training for free! In *Advances in Neural Information Processing Systems (NeurIPS)*.

Steinhardt, J., Koh, P. W., and Liang, P. (2017). Certified defenses for data poisoning attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tran, B., Li, J., and Madry, A. (2018). Spectral signatures in backdoor attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tsipras, D., Santurkar, S., Engstrom, L., Turner, A., and Madry, A. (2019). Robustness may be at odds with accuracy. In *International Conference on Learning Representations (ICLR)*.

Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., and Zhao, B. Y. (2019). Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 707–723.

Wang, Z., Pang, T., Du, C., Lin, M., Liu, W., and Yan, S. (2023). Better diffusion models further improve adversarial training. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 36246–36263.

Wong, E. and Kolter, J. Z. (2018). Provable defenses against adversarial examples via the convex outer adversarial polytope. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5286–5295.

Wong, E., Rice, L., and Kolter, J. Z. (2020). Fast is better than free: Revisiting adversarial training. In *International Conference on Learning Representations (ICLR)*.

Xu, W., Evans, D., and Qi, Y. (2018). Feature squeezing: Detecting adversarial examples in deep neural networks. In *Network and Distributed System Security Symposium (NDSS)*.

Yin, D., Chen, Y., Kannan, R., and Bartlett, P. (2018). Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5650–5659.

Zhang, H., Yu, Y., Jiao, J., Xing, E., El Ghaoui, L., and Jordan, M. (2019). Theoretically principled trade-off between robustness and accuracy. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 7472–7482.